

1 Phase 2 – Divide and Conquer + Greedy Approaches

1.1 Introduction

The Twiddle Puzzle is a rotation-based $N \times N$ grid puzzle in which each move rotates a 2×2 sub-grid counter-clockwise. The objective is to transform the current board into a sorted goal configuration.

In Phase 2, we were required to solve the puzzle using:

- Divide and Conquer (D&C)
- Greedy strategy

Unlike BFS and A*, which guarantee a solution under finite state space, Divide and Conquer approaches do not guarantee completeness. We implemented three D&C-inspired strategies and analyzed their behavior.

1.2 1. Spatial Divide and Conquer

1.2.1 Core Idea

The board is recursively divided into smaller spatial subregions until 2×2 blocks are reached. Each 2×2 block corresponds to a valid rotation move.

At every recursion:

- **Divide:** Partition region into smaller subregions
- **Conquer:** Evaluate each 2×2 rotation
- **Combine:** Choose rotation with minimum heuristic value

1.2.2 Algorithmic Inspiration

Derived from:

- Classical Divide and Conquer (similar to Merge Sort)
- Greedy best-choice strategy
- Heuristic local optimization

1.2.3 Pseudocode

1.2.4 Time Complexity

Recurrence relation:

$$T(n) = 4T(n - 1) + O(1)$$

Worst-case complexity:

$$O(4^N)$$

Practical complexity is reduced due to pruning.

Algorithm 1 Spatial Divide & Conquer

```
1: function SPATIALDC(matrix, s1, s2, e1, e2)
2:   if region size is  $2 \times 2$  then
3:     next  $\leftarrow$  rotate(matrix)
4:     h  $\leftarrow$  heuristic(next)
5:     return (move, h)
6:   end if
7:   children  $\leftarrow \emptyset$ 
8:   Add recursive calls on 4 subregions to children
9:   Remove null children from children
10:  Sort children by heuristic value
11:  return child with minimum heuristic
12: end function
```

1.2.5 Space Complexity

$$O(N^2)$$

1.2.6 Limitations

- Local greedy decisions
- No global planning
- May oscillate
- No convergence guarantee

1.3 2. Cycle-Based Divide and Conquer

1.3.1 Core Idea

The board configuration is treated as a permutation of numbers 1 to N^2 . Every permutation can be decomposed into disjoint cycles.

Steps:

1. Decompose permutation into cycles
2. Select largest incorrect cycle
3. Evaluate all legal moves
4. Choose move reducing cycle mismatch

1.3.2 Algorithmic Inspiration

Derived from:

- Cycle Decomposition in Group Theory

- Graph Cycle Detection
- Greedy Local Search

1.3.3 Pseudocode

Algorithm 2 Cycle Divide & Conquer

```

1: function CYCLEDG(board)
2:   cycles  $\leftarrow$  DecomposePermutation(board)
3:   largest  $\leftarrow$  cycle with maximum length
4:   bestScore  $\leftarrow \infty$ 
5:   for each legal move m do
6:     next  $\leftarrow$  apply(m)
7:     score  $\leftarrow$  cycleMismatch(next) + globalHeuristic(next)
8:     if score < bestScore then
9:       bestScore  $\leftarrow$  score
10:      bestMove  $\leftarrow$  m
11:    end if
12:  end for
13:  return bestMove
14: end function

```

1.3.4 Time Complexity

Cycle decomposition:

$$O(N^2)$$

Move evaluation:

$$(N - 1)^2 \text{ moves}$$

Each evaluation costs:

$$O(N^2)$$

Total complexity:

$$O(N^4)$$

1.3.5 Space Complexity

$$O(N^2)$$

1.3.6 Limitations

- Rotations affect multiple cycles
- Cannot isolate cycles perfectly
- May oscillate without global memory

1.4 3. Depth-Based Divide and Conquer

1.4.1 Core Idea

Instead of dividing space or permutation, we divide the search tree by depth.
At each level:

- **Divide:** Generate child states
- **Conquer:** Recursively evaluate subtree
- **Combine:** Select branch with minimum heuristic

1.4.2 Algorithmic Inspiration

Derived from:

- Depth-Limited Search
- Branch and Bound

1.4.3 Pseudocode

Algorithm 3 Depth Divide & Conquer

```
1: function DEPTHDC(board, depth)
2:   if depth = 0 OR board solved then
3:     return heuristic(board)
4:   end if
5:   best  $\leftarrow \infty$ 
6:   for each legal move m do
7:     next  $\leftarrow$  apply(m)
8:     score  $\leftarrow$  DepthDC(next, depth - 1)
9:     best  $\leftarrow$  min(best, score)
10:  end for
11:  return best
12: end function
```

1.4.4 Time Complexity

Let:

$$b = (N - 1)^2 \quad (\text{branching factor})$$

$$d = \text{LOOKAHEAD}$$

Time complexity:

$$O(b^d)$$

1.4.5 Space Complexity

$$O(d \cdot N^2)$$

1.4.6 Limitations

- Limited lookahead
- No guarantee of optimality
- May miss long-term improvements

1.5 Loop Prevention Enhancement

To prevent oscillation, we introduced:

- Global visited state memory
- Dominance pruning

Inspired by:

- Memoization
- Branch and Bound

1.6 Comparison

Method	Inspired From	Time Complexity	Complete	Optimal
Spatial D&C	Recursive Partitioning	$O(4^N)$	No	No
Cycle D&C	Permutation Cycles	$O(N^4)$	No	No
Depth D&C	Depth-Limited Search	$O(b^d)$	No	No
BFS	Level-order Search	$O(b^d)$	Yes	Yes
A*	Best-first Search	$O(b^d)$	Yes	Yes

1.7 Conclusion

Spatial, Cycle, and Depth approaches demonstrate different interpretations of Divide and Conquer:

- **Spatial:** Divide by region
- **Cycle:** Divide by permutation structure
- **Depth:** Divide by search depth

All combine recursive subdivision with greedy selection. Unlike BFS and A*, these methods reduce exploration but sacrifice completeness and optimality.

They highlight trade-offs between computational cost, memory usage, and convergence guarantees.